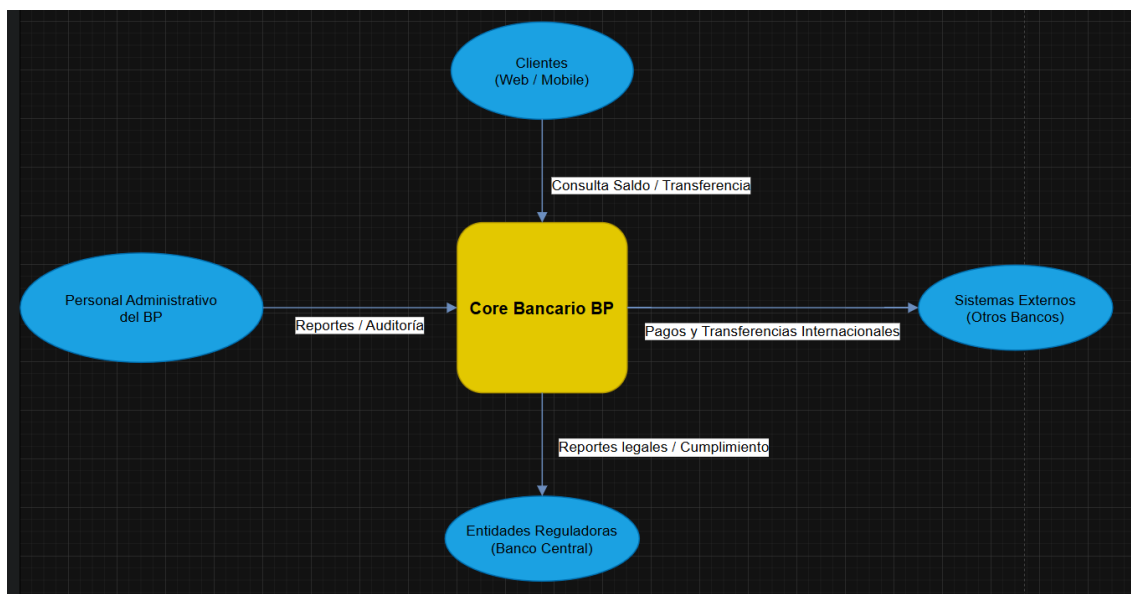


EJERCICIO PRÁCTICO – RICARDO RANGLES

A continuación, se presenta la solución práctica para este ejercicio práctico.

1. DIAGRAMA DE CONTEXTO



Justificación del Diagrama de Contexto para la entidad BP

El diagrama de contexto presenta una visión de alto nivel de todo el ecosistema del Core Bancario del BP, mostrando cómo interactúa con sus actores y sistemas externos, permitiendo identificar los puntos críticos de seguridad, cumplimiento normativo y disponibilidad.

Actores Externos

- **Clientes (Web y Mobile):** Son los usuarios finales que realizan operaciones como consultas de saldo, transferencias y pagos. Su inclusión permite visualizar la necesidad de interfaces seguras y confiables, así como un canal de comunicación cifrado y alta disponibilidad para garantizar la continuidad del servicio.
- **Personal Administrativo del Banco:** Son los operadores internos del banco encargados de la gestión de cuentas, aprobaciones y reportes. La interacción con el Core Bancario indica la importancia de controles internos y segregación de funciones (roles), alineado con normativas de auditoría y control financiero.

- **Sistemas Externos (Otros Bancos):** Son las conexiones con entidades externas para transacciones interbancarias. Esto evidencia la necesidad de APIs seguras, control de flujo de datos y tolerancia a fallos en los procesos de pago y reconciliación.
- **Entidades Reguladoras:** El banco BP tiene la obligación de entregar reportes legales, cumplir con normativas y garantizar la trazabilidad de las operaciones a entidades como Banco Central y Contraloría. Incluye la Ley de Protección de Datos y requerimientos de auditoría y conservación de registros.

Sistema Principal: Core Bancario

El Core Bancario se representa como un sistema centralizado, aunque internamente está compuesto por microservicios que se detallarán en el diagrama de contenedores.

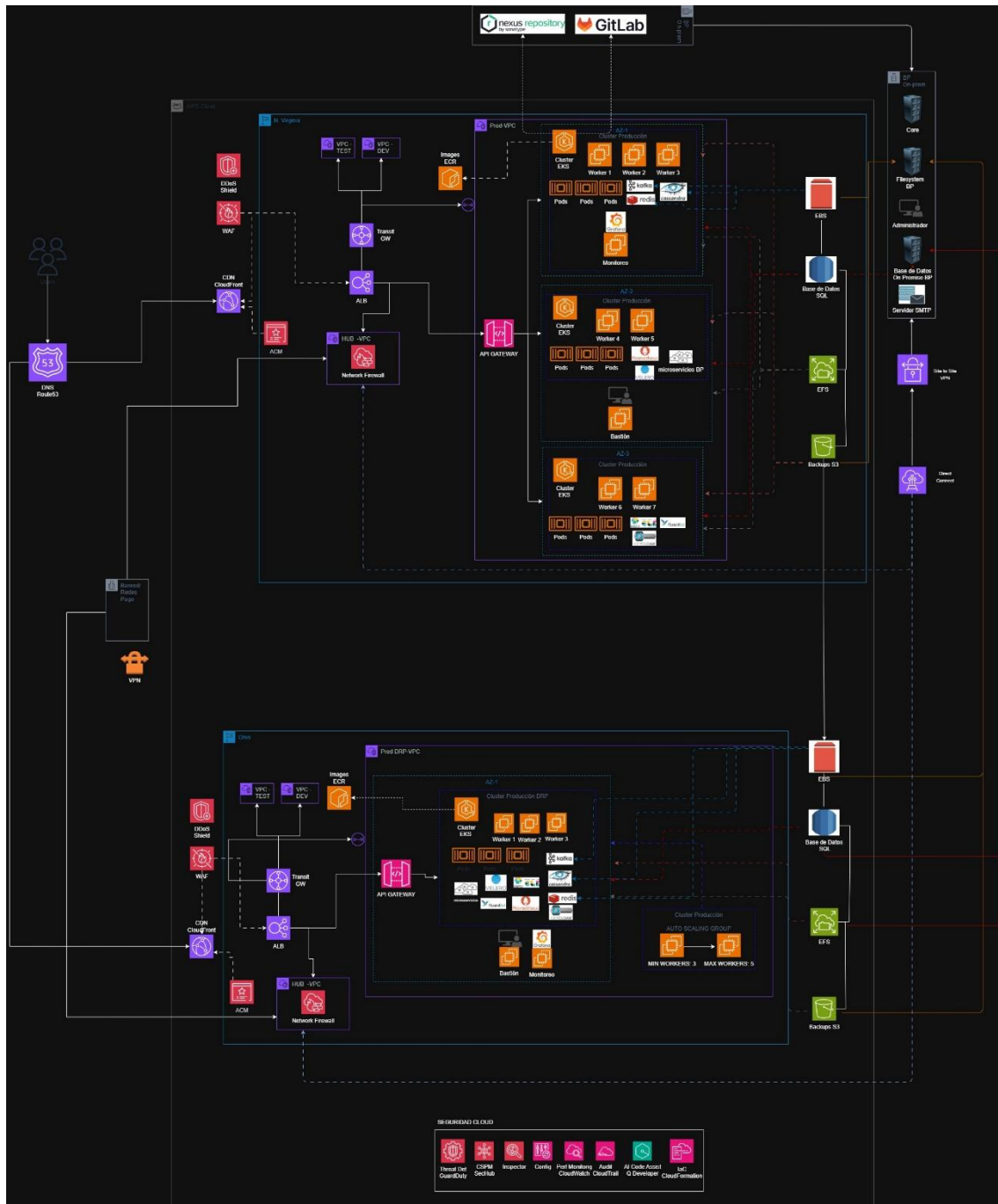
Este sistema centraliza todas las operaciones, gestiona la base de datos financiera y sirve como intermediario para las interacciones entre usuarios internos, clientes y sistemas externos.

Interacciones y Flujo de Información

- Clientes → Core Bancario: Hace las solicitudes de consultas y transacciones.
- Personal Administrativo del Banco Bancario → Core Bancario: envío de reportes y auditorías.
- Core Bancario → Sistemas Externos: pagos y transferencias internacionales.
- Core Bancario → Regulador: entrega de reportes legales, cumplimiento y trazabilidad, asegurando la transparencia frente a entidades reguladoras.
- Seguridad, Cumplimiento y Alta Disponibilidad.
- Seguridad de datos: todas las interacciones se cifran, y el acceso se controla mediante roles y autenticación centralizada.
- Cumplimiento normativo: el diagrama de contexto destaca los puntos donde se aplican leyes de protección de datos, regulaciones financieras y requerimientos de auditoría.

- Disponibilidad y tolerancia a fallos: el Core Bancario actúa como nodo central de operaciones críticas, diseñado para ser replicable y escalable en el diagrama de contenedores, garantizando continuidad en caso de fallos.

2. DIAGRAMA DE CONTENEDORES



Para visualizar mejor el diagrama estará subido dentro del repositorio Ejercicio-Practic-Devsu

<https://github.com/RicardoRangles/Ejercicio-Practico-Devsu.git>

Justificación del Diagrama de Contenedores para la entidad BP

A continuación, se detalla la justificación de todo lo que contempla el diagrama de contenedores:

1. Justificación de la Arquitectura Bancaria BP en AWS / EKS

- **Alta disponibilidad (HA) y tolerancia a fallos:** EKS permite distribuir los pods y microservicios críticos entre múltiples nodos y zonas de disponibilidad, garantizando continuidad del negocio ante fallos de infraestructura.
- **Seguridad integral:** se implementa protección frente a amenazas externas e internas usando TLS gestionado por ACM, control de acceso centralizado con Keycloak (OAuth 2.0 y RBAC), firewall de red, WAF, Shield y segmentación mediante Transit Gateway. Los datos se cifran en tránsito y en reposo, cumpliendo con normativas financieras y de protección de datos.
- **Observabilidad completa:** mediante Prometheus, Grafana, Zabbix, Fluent Bit y ELK Stack, se monitorean métricas, logs y eventos, generando alertas tempranas y trazabilidad para auditoría y cumplimiento normativo.
- **Recuperación ante desastres (DR):** replicación de bases de datos críticas (Cassandra, Redis, RDS/SQL), backups automáticos con Velero y failover multi-región mediante Route 53, asegurando que los servicios permanezcan disponibles ante fallos regionales.
- **Portabilidad y flexibilidad:** los servicios dentro de EKS (Redis, Kafka, Cassandra, Keycloak, Velero) son independientes de la nube, facilitando futura migración a entornos on-premise si fuese necesario.
- **Optimización de costos:** se usan servicios gestionados de AWS cuando agregan valor (EKS, ALB, ACM, S3, CloudFront), y la replicación se limita a componentes críticos para equilibrar disponibilidad y eficiencia económica.

Se escogió EKS debido a su capacidad de orquestación de contenedores, escalabilidad automática y gestión de nodos, permitiendo desplegar microservicios críticos y bases de datos distribuidas con alta disponibilidad.

Servicios de AWS como ALB, CloudFront, WAF, Shield y Route 53 aseguran que la arquitectura sea segura, resiliente y eficiente, mientras que los componentes de observabilidad y CI/CD on-premise permiten auditoría y control total del ciclo de vida del software.

2. Justificación por componente y paquete tecnológico

Componente / Servicio	Función / Justificación	Tecnología / Consideraciones
API Gateway / Ingress Controller	Entrada unificada para clientes web y móvil. Gestiona rutas hacia los microservicios, aplica TLS y políticas de seguridad. Justificación 1: Centraliza la exposición de servicios y simplifica autenticación y control de tráfico. Justificación 2: Facilita escalamiento horizontal y balanceo de carga.	Ingress Controller (ALB opcional para AWS), TLS gestionado por ACM
Redis Clúster	Almacena información temporal y mejora la respuesta de microservicios críticos. Justificación 1: Reduce la carga sobre la base de datos y mejora tiempos de respuesta. Justificación 2: Soporta replicación y HA dentro del clúster (3 réplicas).	Redis Clúster
Kafka Clúster	Orquestación de eventos y desacoplamiento de microservicios. Justificación 1: Tolerancia a fallos y resiliencia de transacciones críticas. Justificación 2: Facilita integración con sistemas externos (SWIFT, otros bancos).	Kafka
Cassandra	Base de datos no transaccional y almacenamiento histórico. Justificación 1: Permite alta disponibilidad y escalabilidad horizontal. Justificación 2: Garantiza persistencia de datos críticos para auditoría y recuperación ante desastres.	Cassandra Clúster

Componente / Servicio	Función / Justificación	Tecnología / Consideraciones
AWS RDS/ SQL	Justificación 1: Proporciona una base de datos transaccional gestionada, con alta disponibilidad y replicación automática, asegurando persistencia de datos críticos y continuidad del negocio. 2) Facilita el cumplimiento normativo mediante backups automáticos, snapshots y cifrado de datos en reposo, simplificando la administración operativa y reduciendo riesgos de fallos humanos.	AWS RDS (MySQL, PostgreSQL o Aurora), multi-AZ, backups automáticos y cifrado en reposo
Keycloak	Gestión de identidad y autorización, OAuth 2.0, onboarding biométrico (huella/facial). Justificación 1: Centraliza autenticación y RBAC, asegura cumplimiento normativo. Justificación 2: Soporta integración con onboarding biométrico (facial, huella) y aplicaciones móviles.	Keycloak HA dentro del clúster
Velero	Backup y recuperación ante desastres. Justificación 1: Permite restaurar aplicaciones críticas y volúmenes persistentes. Justificación 2: Facilita migración a otro entorno o región en caso de falla.	Velero con almacenamiento en S3 / MinIO / ODF
Fluent Bit / ELK Stack	Logging centralizado y análisis de eventos. Justificación 1: Trazabilidad de transacciones, auditoría y detección de incidentes; ELK para dashboards y búsquedas avanzadas. Justificación 2: Soporta auditoría y cumplimiento de regulaciones de protección de datos.	Fluent Bit DaemonSet + Elasticsearch / Logstash / Kibana
Prometheus / Grafana / Zabbix	Observabilidad y monitoreo del clúster y microservicios. Justificación 1: Métricas del clúster, alertas proactivas, SLA y KPI de disponibilidad. Justificación 2: Provee métricas para escalamiento y auditoría.	Prometheus + Grafana + Zabbix

Componente / Servicio	Función / Justificación	Tecnología / Consideraciones
ACM	Gestión de certificados TLS. Justificación 1: Cifrado extremo a extremo, renovación automática y cumplimiento de normativas de seguridad. Justificación 2: Renovación automática y cumplimiento de estándares de seguridad.	ACM asociado a ALB / Ingress
ALB (Application Load Balancer)	Distribución de tráfico externo a nodos del cluster y alta disponibilidad. Justificación 1: HA entre nodos y AZ. Justificación 2: Facilita failover y manejo de picos de tráfico.	ALB en subnets públicas
Route 53	Failover multi-región, balanceo de tráfico DNS y continuidad ante desastres. Justificación 1: Asegura continuidad del servicio. Justificación 2: Permite DR sin intervención manual.	DNS con health checks y failover
AWS Shield / WAF	Protección frente a ataques DDoS y filtrado de tráfico malicioso. Justificación1: Seguridad de capa 7 y disponibilidad garantizada. Justificación 2: Permite aplicar políticas de seguridad personalizadas por aplicación o ruta, cumpliendo estándares regulatorios y reduciendo riesgos operativos.	Shield Standard + WAF con reglas personalizadas
CloudFront (CDN)	Distribución de contenido estático y dinámico. Justificación 1: Mejora performance global de SPA y app móvil, protege origen y reduce latencia. Justificación 2: Protege el origen de ataques y tráfico malicioso, integrándose con WAF y TLS, asegurando seguridad y optimización del rendimiento de SPA y apps móviles.	AWS CloudFront

Componente / Servicio	Función / Justificación	Tecnología / Consideraciones
Network Firewall / Transit Gateway	Control centralizado de tráfico entre VPCs y on-premise. Justificación 1: Segmentación, cumplimiento y DR entre regiones. Justificación 2: Facilita la arquitectura de recuperación ante desastres multi-región y centralización de políticas de seguridad, reduciendo riesgos y complejidad operativa.	AWS Network Firewall + Transit Gateway
ECR / Imágenes de contenedor	Repositorio seguro de imágenes de contenedor. Justificación 1: CI/CD integrado, control de versiones y seguridad de imágenes. Justificación 2: Facilita el cumplimiento de políticas corporativas de seguridad y gobierno de TI al centralizar el almacenamiento de imágenes y evitar uso de repositorios externos no controlados.	AWS ECR
EBS / EFS / S3	Persistencia de datos de bases de datos y almacenamiento de backups. Justificación 1: EBS para volúmenes de alta IOPS, EFS para compartición de archivos, S3 para backups y logs inmutables. Justificación 2: Permite replicación y escalabilidad según demanda, asegurando DR y disponibilidad de datos, optimizando costos operativos.	AWS EBS, EFS, S3
Site-to-Site VPN / Direct Connect	Conexión segura entre on-premise y nube. Justificación 1: CI/CD (GitLab / Nexus) y futuras migraciones on-premise, tráfico cifrado. Justificación 2: Facilita migración híbrida de workloads, integración de servicios y baja latencia entre entornos, permitiendo operaciones críticas sin interrupción.	VPN + AWS Direct Connect
CI/CD On-Premise	Gestión de pipelines, builds y repositorios de artefactos.	GitLab + Nexus Sonatype

Componente / Servicio	Función / Justificación	Tecnología / Consideraciones
	Justificación 1: permite pruebas, control de versiones y despliegues independientes de la nube. Justificación 2: Garantiza independencia de la nube, reproducibilidad de despliegues y trazabilidad completa de versiones.	

3. Justificación de Aplicaciones frontales

- **SPA Web (React / Angular)**

framework multiplataforma, rápido, integración directa con API Gateway y OAuth 2.0.

- **Móvil (Flutter / React Native):**

multiplataforma, integración con biometría y onboarding facial/huella, consumo seguro de APIs.

Ambas se comunican con el API Gateway/Ingress y autenticación centralizada en Keycloak, asegurando cumplimiento normativo y seguridad de extremo a extremo.

4. Distribución de nodos y pods

- Cluster AKS de 6-7 nodos, distribuido por zonas de disponibilidad.
- Microservicios críticos: replicados 2-3 veces en nodos distintos.
- Kafka, Redis y Cassandra: 3 réplicas distribuidas para garantizar HA y tolerancia a fallos.
- Observabilidad y logging: Prometheus, Grafana, Zabbix y Fluent Bit desplegados en todo el clúster.
- Velero: backups programados hacia S3 / EFS / MinIO, replicación en nodos designados.

Esto asegura resiliencia, balanceo de carga, tolerancia a fallos y cumplimiento de SLA.

5. Justificación de Seguridad y cumplimiento

- TLS / ACM: cifrado de extremo a extremo.
- OAuth 2.0 / Keycloak: autenticación y control de acceso centralizado, onboarding biométrico.
- DDoS / WAF / CloudFront: protección de capa 7, mitigación de ataques y CDN global.
- Network Firewall / Transit Gateway: segmentación de tráfico y seguridad de red.
- Logs y auditoría (ELK + Fluent Bit): trazabilidad completa, cumplimiento normativo.
- Backups (Velero + S3 / EFS / EBS): recuperación ante desastres y consistencia de datos.

6. Justificación Observabilidad y Monitoreo

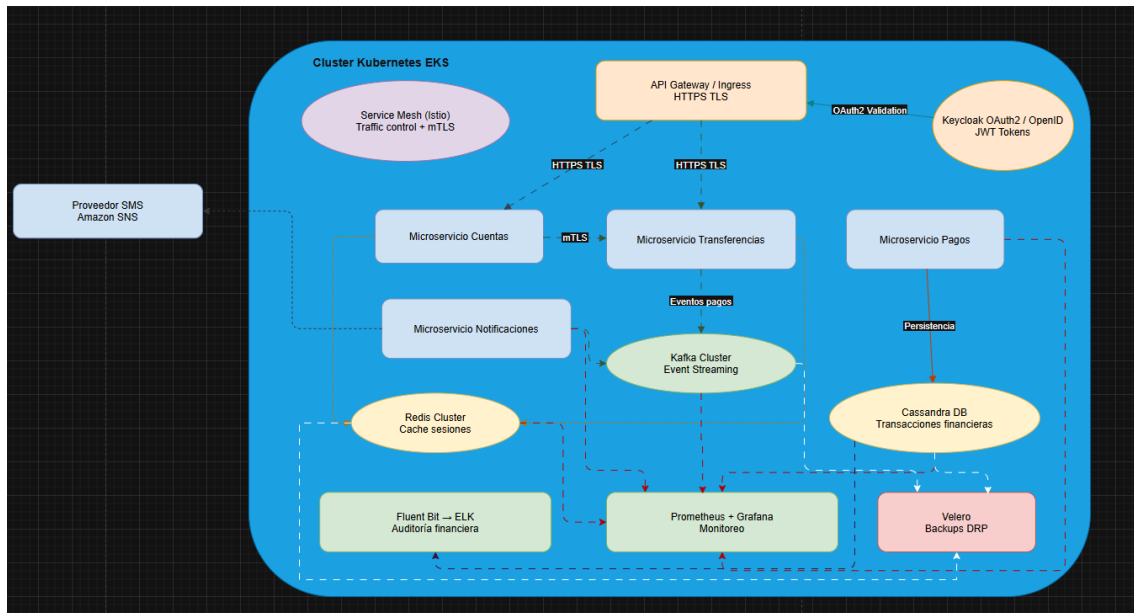
- Prometheus / Grafana: métricas de pods, nodos y microservicios.
- Zabbix: alertas de SLA, disponibilidad y capacidad.
- ELK Stack: análisis de logs y auditoría de transacciones.

Esto permite detección temprana de fallos, optimización de rendimiento y cumplimiento de normas financieras.

7. Justificación de Costos

- Uso de servicios gestionados (ACM, ALB, ECR, S3, CloudFront) para reducir costos operativos y tiempo de gestión.
- Replicación solo de componentes críticos para balancear HA vs costo.
- Cluster EKS con nodos escalables horizontalmente para optimizar uso de recursos según demanda.
- Offload de contenido estático a CloudFront y S3 para reducir carga de cómputo.

3. DIAGRAMA DE COMPONENTES



Justificación del Diagrama de Componentes para la entidad BP

El diagrama de componentes presenta la estructura interna del Core Bancario desplegado sobre un clúster de Kubernetes gestionado mediante Amazon EKS, mostrando el nivel de detalle de los microservicios, los mecanismos de comunicación entre ellos y los componentes de soporte necesarios para garantizar seguridad, observabilidad, resiliencia y cumplimiento normativo.

El objetivo de este nivel de arquitectura es evidenciar cómo cada funcionalidad del sistema bancario se implementa mediante microservicios independientes, desacoplados y altamente disponibles, permitiendo escalar de forma horizontal y garantizar tolerancia a fallos.

Arquitectura basada en microservicios

El núcleo del sistema está compuesto por microservicios especializados como cuentas, transferencias, pagos y notificaciones. Cada uno encapsula una funcionalidad específica y se comunica mediante interfaces bien definidas.

Esta separación funcional permite:

- Desacoplamiento entre capacidades del negocio.
- Escalabilidad independiente según la carga de cada servicio.

- Despliegues y actualizaciones sin afectar al sistema completo.

Los microservicios se comunican mediante dos patrones complementarios:

Comunicación Síncrona

Se utiliza para operaciones que requieren respuesta inmediata, por ejemplo, la validación de saldo antes de ejecutar una transferencia.

Comunicación asíncrona basada en eventos

Los servicios publican eventos en Apache Kafka, permitiendo que otros servicios reaccionen sin generar dependencias directas. Este patrón mejora la resiliencia del sistema y facilita la escalabilidad del procesamiento de eventos financieros.

Seguridad y control de acceso

La autenticación y autorización del sistema se gestionan mediante Keycloak, que implementa estándares abiertos como OAuth 2.0 y OpenID Connect.

Este componente permite:

- Autenticación centralizada de usuarios.
- Generación de tokens JWT para acceso seguro a los microservicios.
- Implementación de control de acceso basado en roles (RBAC).

Gestión de datos y rendimiento

Para optimizar el rendimiento del sistema se utiliza Redis como capa de cache distribuida. Redis permite reducir la carga sobre las bases de datos principales almacenando temporalmente información como sesiones, consultas frecuentes y tokens de autenticación.

El almacenamiento principal de las transacciones financieras se gestiona mediante Apache Cassandra, una base de datos distribuida diseñada para soportar grandes volúmenes de datos con alta disponibilidad y tolerancia a fallos.

Esta combinación permite mantener:

- alta velocidad de acceso a datos,
- replicación entre nodos,
- consistencia adecuada para operaciones financieras.

Arquitectura orientada a eventos

El uso de Apache Kafka permite implementar una arquitectura orientada a eventos, donde los servicios publican eventos de negocio que pueden ser consumidos por otros componentes del sistema.

Por ejemplo, cuando se ejecuta una transferencia, el servicio correspondiente publica un evento que puede ser consumido por el servicio de notificaciones, sistemas de auditoría o módulos de análisis de fraude.

Este enfoque permite:

- desacoplar servicios
- mejorar la resiliencia
- procesar eventos financieros de forma escalable

Observabilidad y monitoreo

Para garantizar la operación continua del sistema se implementa una plataforma de observabilidad basada en Prometheus y Grafana, que permiten recolectar métricas del sistema, generar alertas proactivas y visualizar indicadores clave de rendimiento.

Adicionalmente, los logs del sistema se centralizan mediante Fluent Bit, que envía la información hacia una plataforma de análisis basada en Elasticsearch, permitiendo auditoría, trazabilidad de transacciones y análisis forense ante incidentes.

Respaldo y recuperación ante desastres

Para garantizar la continuidad del negocio, los recursos del clúster se respaldan utilizando Velero, que permite realizar copias de seguridad de los recursos de Kubernetes y de los volúmenes persistentes.

Esto facilita la restauración rápida de aplicaciones y datos ante incidentes operativos o desastres, cumpliendo con los requerimientos de recuperación establecidos para sistemas financieros críticos.